

Client Item Selector — Build Specification

Purpose: A single-page web interface where clients can select items (products/services), configure options, and instantly see the calculated price. This file is intended to be handed to a developer (Cursor) as a complete spec: UI, data model, API requirements, acceptance criteria, and clear tags marking **required** and **optional** functionality.

1) Project Overview

- The page is a responsive, accessible UI where users (clients) pick items, choose variants/add-ons, set quantities, and immediately see subtotal, taxes, discounts, and total.
 - The page must be embeddable into the main website and able to post a quote request (including selected line items) to the backend.
-

2) Goals

- Fast, clear selection flow for non-technical clients.
 - Transparent pricing calculations (line-item price, options, taxes, discounts).
 - Easy to convert selections into a saved quote or checkout flow.
-

3) User Stories

1. As a client, I can browse a list of items and add them to the selection cart.
 2. As a client, I can choose variants (size, color, warranty) and optional add-ons per item.
 3. As a client, I can set quantity for each item and see dynamic price updates.
 4. As a business admin, I can fetch the item catalog and pricing rules via API.
 5. As a client, I can share/save my selection as a quote or proceed to checkout.
-

4) UI Layout (desktop first)

- **Header:** breadcrumb, page title (e.g., "Build Your Quote"), help icon.
- **Left pane / Catalog:** searchable list of items with thumbnail, short description, base price, and "Add" button.
- **Main pane / Item details:** when an item is selected, show full details, variants, option toggles, quantity selector, and "Add to selection".
- **Right pane / Quote summary (sticky):** list of selected items, per-item breakdown, subtotal, taxes, discount code input, total, and action buttons: , , .
- **Footer:** terms, contact.

Mobile: Catalog collapses; summary is a bottom drawer or floating summary bar. Quantity controls easily tappable.

5) Data Model (suggested)

```
Item {
  id: string,
  title: string,
  sku?: string,
  thumbnailUrl?: string,
  description?: string,
  basePrice: number, // in smallest currency unit or decimal
  currency: string, // e.g., "INR"
  variants?: [
    { id: string, name: string, priceDelta: number }
  ],
  options?: [
    { id: string, name: string, type: "checkbox"|"radio"|"select",
      priceDelta: number }
  ],
  taxCategory?: string, // to apply tax rules
  available?: boolean
}

QuoteLineItem {
  itemId: string,
  variantId?: string,
  options: [{optionId, value}],
  unitPrice: number,
  quantity: number,
  lineTotal: number
}

Quote {
  id?: string,
  customerName?: string,
  email?: string,
  phone?: string,
  items: [QuoteLineItem],
  subtotal: number,
  taxes: [{name, amount}],
  discount?: {type: "percent"|"flat", value: number},
  total: number,
```

```
    createdAt?: string  
  }
```

6) API Endpoints (developer to implement / integrate)

- `GET /api/items` — Returns catalog with variants & options. [REQUIRED]
- `GET /api/items/:id` — Item detail. [OPTIONAL for performance if `GET /api/items` returns full data]
- `POST /api/quotes` — Create and save a quote (payload → Quote). Returns quote id & PDF link. [REQUIRED]
- `POST /api/quote/calc` — (Optional) Accepts selection and returns server-side calculation (tax, discounts) for verification. [OPTIONAL]
- `POST /api/discount/validate` — Validate coupon/discount codes. [OPTIONAL]

Security: all write endpoints must be authenticated via bearer token (or a time-limited frontend token) where applicable. If quotes are anonymous, create `POST /api/quotes/guest` with CAPTCHA throttling. [REQUIRED]

7) Pricing Rules & Calculation Logic

1. **Line unit price** = $\text{basePrice} + \text{variant.priceDelta} + \text{sum}(\text{option.priceDelta})$.
2. **Line total** = unit price × quantity.
3. **Subtotal** = sum(line totals).
4. **Tax**: support multiple tax lines (e.g., CGST/SGST or VAT). Tax rules apply per item based on taxCategory. Tax calculations must be visible. [REQUIRED]
5. **Discounts**: flat or percentage. If both item-level and cart-level discounts apply, define precedence (item-level first, then cart-level). [OPTIONAL]
6. **Rounding**: final total rounded to 2 decimal places (currency smallest unit). Show intermediate precision where helpful. [REQUIRED]

Client-side calculation should mirror server logic for instant feedback, but final authoritative calculation must be performed server-side on `POST /api/quotes` to prevent tampering. [REQUIRED]

8) Functional Requirements (Module-wise with tags)

Catalog & Selection

- [REQUIRED] Display items with thumbnail, name, base price.
- [REQUIRED] Add items to selection and set quantity.
- [OPTIONAL] Search, filter, and category browsing.

Item Configuration

- [REQUIRED] Variants (single-select) and options (checkboxes/multi-select) that affect price.
- [OPTIONAL] Image gallery & zoom for product detail.

Quote Summary & Actions

- [REQUIRED] Live-updating quote summary showing line breakdown, subtotal, tax, discounts, total.
- [REQUIRED] `Save Quote` — posts to `POST /api/quotes` and returns quote id.
- [OPTIONAL] `Download PDF` — server returns generated PDF of quote.
- [OPTIONAL] `Email Quote` — send quote to customer email.

Validation & UX

- [REQUIRED] Prevent adding unavailable items (show out-of-stock badge).
- [REQUIRED] Quantity must be integer ≥ 1 and within available stock (if stock managed).
- [OPTIONAL] Suggest similar items when an item is unavailable.

Security & Performance

- [REQUIRED] Rate-limit `POST /api/quotes` (anti-spam), require CAPTCHA for anonymous calls.
- [OPTIONAL] Implement optimistic local caching of catalog to improve speed.

Accessibility

- [REQUIRED] Keyboard navigable, screen-reader friendly labels, aria attributes, proper color contrast.

9) UI States & Edge Cases

- Empty selection state (CTA to browse or contact sales). [REQUIRED]
- Network error state with retry. [REQUIRED]
- Server-side price mismatch: if server returns different total after `POST /api/quotes`, show a clear message: "Prices updated; please review changes" and display delta adjustments. [REQUIRED]

10) Acceptance Criteria (what 'done' looks like)

1. User can add at least 10 catalog items, configure options, and see total update instantly.
 2. `POST /api/quotes` returns a quote id and saved data matches client selection (server-side verification). [REQUIRED]
 3. Quote PDF can be downloaded (if optional module enabled).
 4. Page passes basic accessibility checks (tab order, alt text, aria labels). [REQUIRED]
 5. No JavaScript console errors for major flows; mobile behavior intact. [REQUIRED]
-

11) Assets & Handoff

- Provide thumbnails (min 400×400), SVG icons for quantity controls, and a style token file (colors, fonts, spacing).
 - Provide sample API response JSON for items (so Cursor can mock during dev).
-

12) Suggested Mock Data (example)

```
{
  "items": [
    {
      "id": "i1", "title": "Product A", "basePrice": 25000,
      "currency": "INR", "variants": [
        {
          "id": "v1", "name": "2-year warranty",
          "priceDelta": 1500
        }
      ], "options": [
        {
          "id": "o1", "name": "Installation",
          "priceDelta": 500
        }
      ], "available": true
    }
  ]
}
```

13) Testing Checklist

- Add item, change variant/options, change quantity → totals match expected calculation.
 - Apply discount code (if enabled) → total updates and discount validated server-side.
 - Simulate server price change and confirm mismatch flow.
 - Save quote and verify backend record contains full line-item detail.
-

14) Notes for Cursor (developer)

- You can implement the frontend as a single React page (Next.js preferred). Keep the quote summary isolated (e.g., `QuoteSummary` component) so it can be embedded elsewhere.
 - Use the provided sample API JSON to build mock server during implementation.
 - Respect the [REQUIRED]/[OPTIONAL] tags above and flag any trade-offs if a required item would significantly increase complexity.
-

15) How to share & use this file

- Share this document with Cursor. They can use this as the scope-of-work for building the page.
 - If Cursor needs, I can also generate a starter React component (Next.js + Tailwind) wired to mock data.
-

End of spec.